Assignment 1: Simulation of Bodily Movements

Derivation and simulation of motion equations

Deadline: 17:45h Friday March 14th 2025

In this assignment you will build a simple model of human locomotion and use it to simulate several motion patterns. The goal is to learn in a hands-on manner how to use motion equations in 3D and how to perform forward dynamic simulations. The assignment consists of two parts:

I. Derivation of the motion equations and implementation into Simulink.
II. Several simulation exercises.

Write a brief report, answering the questions in part A-D. Furthermore, hand in a zipped file with the Matlab files you used for part A-D. This zip archive contains four folders, named 'partA', 'partB', 'partC' and 'partD'. Each folder contains a file runme.m which:

1. derives the motion equations (do not send the files that were created using the diary -command)
2. runs your Simulink model
3. presents an animation of the results for that particular part.

Some guidelines for writing the m-files:

1. It is useful to divide the script file into cells for each question, or section. Cell- separators are %%
2. Use enough comments (%) to describe what has been programmed.
3. Add all your Matlab-code to the report into an appendix.

Goals

After this assignment, students should be able to:

1. Create a biologically-true rigid body linkage that describes human movement.
2. Derive the motion equations using the TMT method.
3. Implement the motion equations into Matlab/Simulink.
4. Apply muscle inputs, torques and rotational stiffness/damping by including them in the differential equation.
5. Interpret the simulated movement and reflect on how it depends on the given inputs.

Important Information

<u>Reports handed in after the deadline will not receive formative feedback</u>

Upload the report of the assignment to Brightspace. The assignments are obligatory and the reports will have to be graded as 'sufficient' in order to get a grade for the course. The assignments help prepare for the computer tests.

© 2025 Alle rechten voorbehouden | TU Delft
Behoudens de in of krachtens de Auteurswet gestelde uitzonderingen, mag niets uit deze uitgave worden verveelvoudigd en/of openbaar gemaakt worden door middel van druk, fotokopie, microfilm, web-publishing of op welke andere wijze dan ook en evenmin in een gegevensopzoeksysteem worden opgeslagen zonder voorafgaande schriftelijke toestemming van de houder van de copyrights.

Working together in groups of two students is allowed. Mention student numbers and names of both students at each page of the report.

In the report you will provide answers (typically figures) to the questions with a character (a-b-c-etc); the bulleted questions are for your own understanding and should not be included in the report.

For the figures, do not forget to add a caption and the units on the axes. When plotting more than one curve in a figure, use different colors and markers. Do not forget to use the correct legends. Choose proper axes scales.

Model

The walking model is illustrated in Fig. 1. It has a straight stance leg and a swing leg consisting of a thigh, shank and foot. The stance foot (front) rotates only around the *z*-axis (medial-lateral), as well as the knee and ankle joints. The hip joint has three DOFs. Angle γ_1 is the angle between the stance leg and the vertical. Angle α_2 is the abduction/adduction angle of the hip, β_2 the medial and lateral rotation of the hip and γ_2 the flexion-extension angle of the hip. Angle γ_3 defines flexion-extension of the knee and γ_4 plantar flexion and dorsi-flexion of the ankle. The masses of the segments (M_{thigh}, M_{shank} and M_{foot}) are modeled as point masses, and the total mass of the head, arms and trunk (M_{HAT}) is modeled as a single point mass between the hips. Rotational inertia is neglected. Muscle actuation is modeled as pure torques around the joints. Some joints may require passive stiffness and/or damping or even active stiffness and/or damping to prevent overstretching.

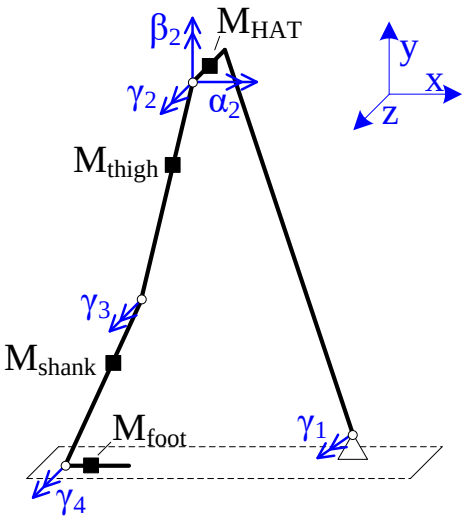

Fig. 1. Side view of the simple walking model. It has two legs, four joints with a total of 6 DOFs and four point masses.

© 2025 Alle rechten voorbehouden | TU Delft

Part I: Motion equations

In this part, you will derive the motion equations of the walking model and implement them into Simulink resulting in a simulation of a walking person, which you will use in Part II. The examples given in the 3dpendulum_examples.zip can be used for the implementation of the walking model. Note that either one holds required features so be sure to inspect them. The files in the folder 'Stumbler_Structure' must be used to simulate the walking person. Only change the lines as indicated in the questions. Do not change other lines!

Symbolic derivation of motion equations

Before deriving the motion equations you will have to dimension the model and make appropriate estimates of the lengths and masses. Then, you can derive the motion equations using the TMT method (see Chapter 3 of the lecture notes "Multibody Dynamics B" on Brightspace). The first step is expressing the positions of the joints and masses in terms of the generalized coordinates (joint angles in our case). The easiest way to do this is to use elementary Euler angles. Use Z-X-Y for the rotation order. The result of this step is the transformation vector T_i. In the second step, you use differentiation, which finally yields the motion equations

$$\bar{\boldsymbol{M}}\ddot{\bar{\boldsymbol{q}}} = \bar{\boldsymbol{f}} \tag{1}$$

Where $\bar{\boldsymbol{M}}$ is the reduced mass matrix, $\boldsymbol{f}$ the reduced force vector and $\ddot{\bar{\boldsymbol{q}}}$ the vector of generalized coordinates.

Follow the steps below to derive the motion equations:

1. Use the files in the folder 'Stumbler_Structure' as a starting point for the Matlab implementation. We will start with the file stumbler_derive_symbolic.m.
2. First, study the file pendulum2_derive_symbolic.m (2seg_5dof) to see how to use Matlab's symbolic toolbox to derive the transformation matrix T_i, its Jacobian (i.e. derivative) and the convective acceleration vector.
3. To implement this for the walking model, first create the symbolic variables in stumbler_derive_symbolic.m .
4. Define the joint and mass locations using the rotation matrices (rotx, roty, rotz) and use Z-X-Y for the rotation order. Also define the state vector and state derivative vector.
5. Derive the transformation matrix T_i, its Jacobian (i.e. derivative) and the convective acceleration vector.
6. In the file pendulum2_derive_symbolic.m, the part involving the diary-command is used to generate a script to numerically compute these vectors/matrices in the S-function later on. Save the symbolic derivation of the walking model using the diary function.

© 2025 Alle rechten voorbehouden | TU Delft

Simulink implementation

Simulink is used to do the numerical integration of the motion equations. You need to implement your model in a S-function.

Hints:

1. Read Chapters 1- 3 of the Matlab manual on S-functions, it's on Brightspace.
2. Download and try all example files. Together, they contain all elements that you need in your model. Readme-files are included in the zip-archive.

Follow the steps below to implement your model in a S-function:

1. Use the file sfun_stumbler.m and define the model masses, lengths and gravity. Use reasonable values (i.e. use the tables of the presentation http://slideplayer.com/slide/4311170/).
1. Fill in the init sizes (lines 105-113) and define the states and state derivatives (lines 134-135).
2. Determine the mass matrix (line 181).
3. Calculate the reduced mass matrix, the force vector, the reduced force vector including stiffness and damping and derivatives qddot (lines 226-237).

A. Perform a single step

After implementation, you will first model a single step of a walking person (part A). The knee of the swing leg is straight and the foot is flat on the ground. The step ends when the swing foot hits the ground again. Be sure that the foot keeps ground clearance throughout the swing!

Follow the steps below:

1. Set the initial conditions of the system such that the angle between stance leg and the vertical is 25° and the angle between the two legs 50° by defining the initial angles and angular velocities. Give the stance leg a sufficient large forward angular velocity as initial condition, such that the hip is able to move beyond its highest point. (lines 115-131)
2. To run the simulation and plot the results, fill in the file plot_stumbler.m.
3. Run the Simulink model to simulate the motion pattern and to plot your results.

This simulation may not result in a natural step. If needed, you can use extra joint torques, damping or stiffness to simulate a proper step. To improve the movement, implement the points below step by step and constantly check the integrity of your model.

1. Include a statement to prevent overstretching of the knee (lines 213-216).
2. Add stiffness and damping to the ankle joint (lines 30-33).
3. Add joint torques in the Simulink model by changing the parameters of the step functions.

Do this step by step, and constantly check the integrity of your model. For instance:

1. Disable gravity and all joint torques: does your model look like a walker?
2. Enable gravity: does your model act like a giant, passive pendulum?

© 2025 Alle rechten voorbehouden | TU Delft

3. Disable gravity and apply a joint torque to each joint (one by one). Does the correct joint start rotating?
4. Add stiffness and damping to certain joints: does this prevent joint motion?

a. Hand in a zipped file with the Matlab files you used for part A containing a folder 'partA'. Include a file runme.m running your scripts and showing your animation.

b. Include a figure with all applied joint torques and joint angles against time. Make sure the lines are recognizable when printed in black and white and include a legend.

- Where did you apply stiffness and/or damping, and in what quantity?
- What joint torques have you used? Are they realistic?
- How long does one step take? Estimate the walking speed. Is it realistic?

Part II: Simulation assignments

When all works in Part I, you can start the simulation exercises of Part II. In this part, you will model how a brick on the road causes the person to trip (part B). To prevent tripping, you will actuate the proper joints in order to step over the brick (part C). Finally, you will simulate an alternative way for a person with a plastered swing leg (part D).

B. Trip over a brick

Add a brick in the path of the swing foot, without changing any other conditions from part A. This causes the person to trip. Think about how you can model the contact between brick and foot.

Follow the steps below:

1. Copy the files of Part A in a folder named Part B.
2. Uncomment the brick dimensions in sfun_stumbler.m.
3. Implement the collision with the brick using the brick dimensions.
4. Calculate the resulting forces on the walking model due to the brick and add these forces to the force vector.
5. Do not change other parts of the script.
6. Uncomment the brick coordinates in plot_stumbler.m and change the script to plot the brick.
7. Change the simulation time in such a way that you can see the total trip.
8. Simulate the movement (do not change the applied joint torques!).

a. Hand in a zipped file with the Matlab files you used for part B containing a folder 'partB'. Include a file runme.m running your scripts and showing your animation.

b. Include a figure of all joint angles against time. Make sure the lines are recognizable when printed in black and white and include a legend.

- At what velocity did the knee / hip hit the floor? Explain.
- How depends this impact velocity on the person's length and weight?

C. Avoid tripping

If the person would have seen the brick, he/she would probably have lifted the foot to avoid tripping over it. Simulate this. Make sure the step looks plausible.

© 2025 Alle rechten voorbehouden | TU Delft

Follow the steps below:

1. Copy the files of Part B in a folder named Part C.
2. Change the applied joint torques in the Simulink model by changing the parameters of the step functions and/or change the stiffness and damping of the knee in sfun_stumbler.m.
3. Do not change other parts of the scripts.
4. Simulate the movement.

a. Hand in a zipped file with the Matlab files you used for part C containing a folder 'partC'. Include a file runme.m running your scripts and showing your animation.

b. Include a figure of all applied joint torques and joint angles against time. Make sure the lines are recognizable when printed in black and white and include a legend.

- Where did you apply torques, why there, and in what quantities?

D. Avoid tripping with a leg in a cast

Our model person recently broke a leg; it is plastered. Simulate the person avoiding a trip with the knee joint locked in extension (apply proper damping and stiffness).

Follow the steps below:

1. Copy the files of Part C in a folder named Part D.
2. Change the joint stiffness and damping to lock the knee joint in extension in sfun_stumbler.m.
3. Change the applied joint torques in the Simulink model by changing the parameters of the step functions to avoid a trip.
4. Do not change other parts of the scripts.
5. Simulate the movement.

a. Hand in a zipped file with the Matlab files you used for part D containing a folder 'partD'. Include a file runme.m running your scripts and showing your animation.

b. Include a figure of all applied joint torques and joint angles against time. Make sure the lines are recognizable when printed in black and white and include a legend.

- Where did you apply torques, why there, and in what quantities?

© 2025 Alle rechten voorbehouden | TU Delft